

Week 5: Human-Computer Interaction - AI Mouse Controller

1. Introduction

This week's assignment focused on extending human silhouette capture and utilizing hand motion and gestures as an interface for human-computer interaction (HCI). The goal was to translate hand landmarks into cursor movements, clicks, and a custom "push" gesture, creating a touchless mouse experience.

2. Methodology

We utilized the **MediaPipe Hands** library for real-time landmark detection and **PyAutoGUI** for controlling the operating system's mouse and keyboard.

2.1. Hand Tracking & Cursor Control

The system tracks the **Index Finger Tip** (Landmark 8). To improve the user experience, we implemented:

- **Sensitivity Scaling (1.5x):** Expands the hand's motion range so that small hand movements can cover the entire screen.
- **Smoothing (Exponential Filter):** Reduces jitter from the camera feed, making cursor movement fluid.
- **Primary Hand Logic:** If two hands are visible, the system prioritizes the one on the left side of the screen, allowing for more stable control in multi-person or complex environments.

2.2. Gesture Dictionary

We defined the following gestures for interaction:

1. **Left Click:** Pinching **Index Finger** and **Thumb**.
2. **Right Click:** Pinching **Middle Finger** and **Thumb**.
3. **Full Screenshot:** Pinching **Ring Finger** and **Thumb**. (Utilizes macOS `screencapture` utility).
4. **Push Gesture (Air Tap):** Moving the hand quickly toward the camera (monitoring Z-depth).

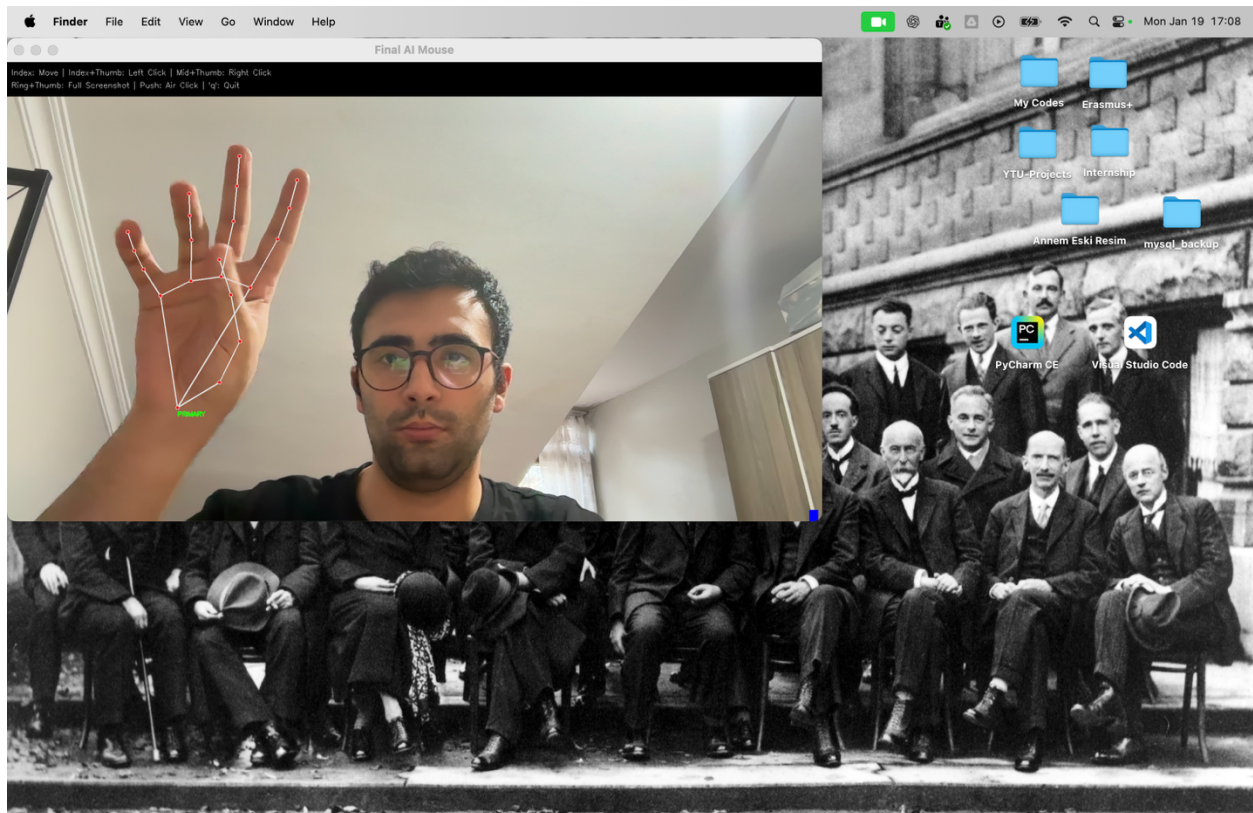
3. Results and Discussion

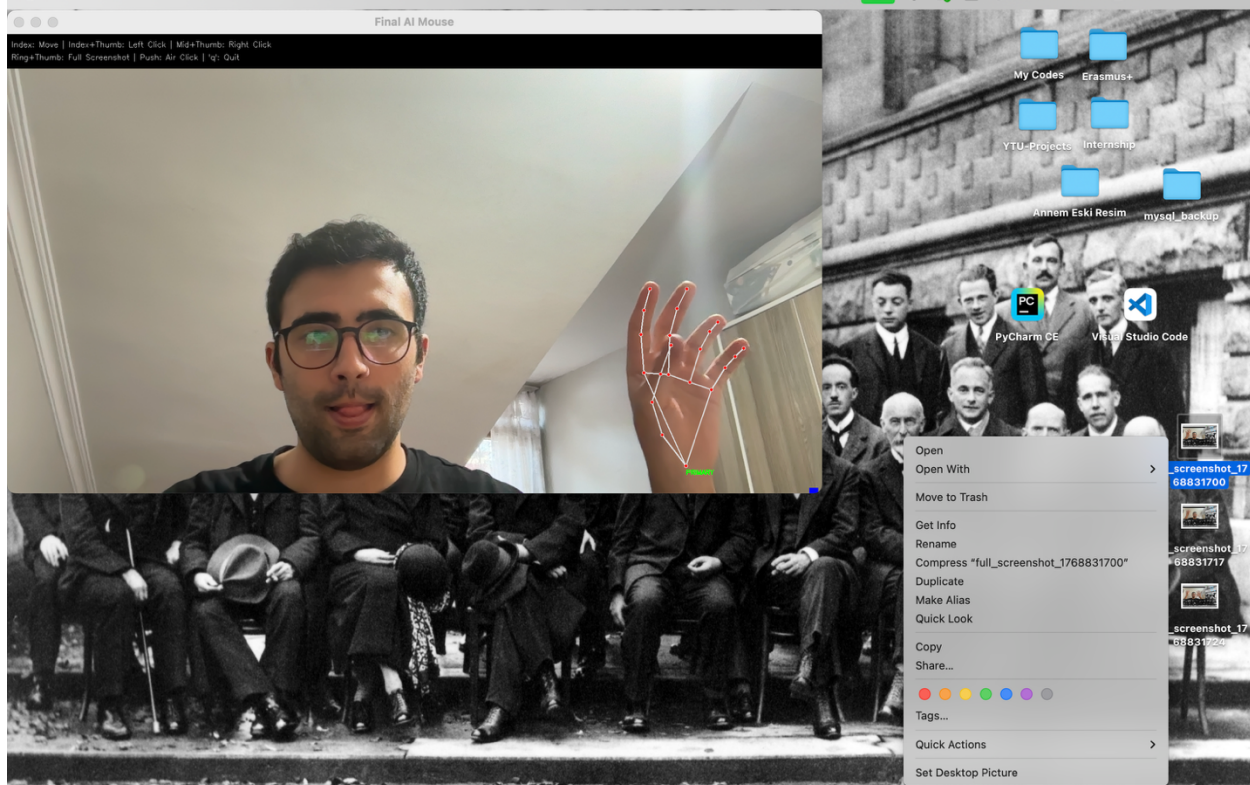
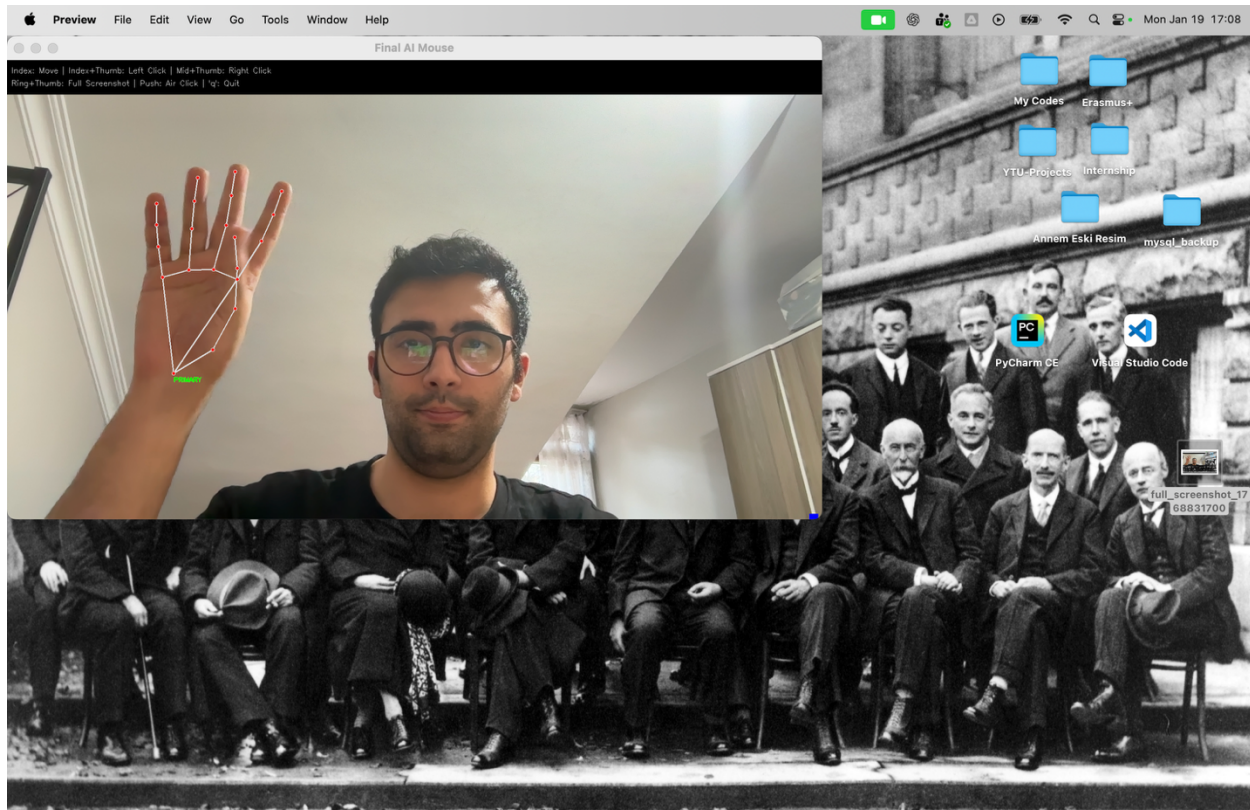
The implementation reached a high level of responsiveness. The addition of a **Screenshot gesture** significantly improved the workflow for documenting the project.

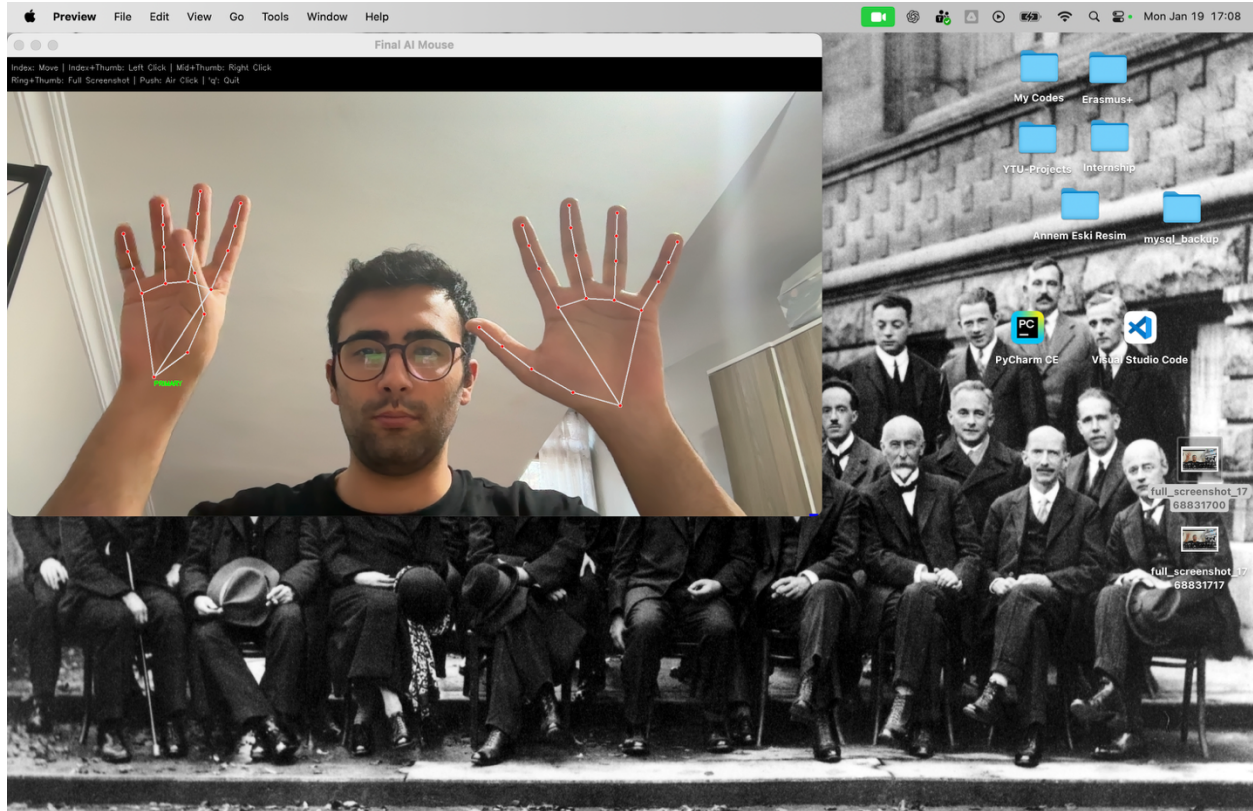
Discussion on Gestures:

- **Pinch Gestures:** Very robust and intuitive. Differentiating between index, middle, and ring pinches worked reliably due to the precision of MediaPipe's landmarking.
- **Push Gesture:** This was the most experimental. The Z-coordinate in MediaPipe is estimated from the hand's size relative to the wrist; while functional, it is more sensitive to hand distance from the camera than XY movement. Using it as a "push to click" requires some practice to avoid accidental triggers.
- **Sensitivity:** The 1.5x scaling was crucial. Without it, the user would have to move their hand across a very large physical space to reach the corners of the screen.

4. Visual Evidence







5. Conclusion

The Week 5 exercises demonstrated that hand-tracking models are sufficiently advanced for daily HCI tasks. By merging all features into a single "Final AI Mouse" script, we created a coherent tool that can replace a physical mouse for basic navigation and interaction.

6. Appendix: Implementation Code

The final integrated script is provided below:

```
# final_ai_mouse.py
import cv2
import mediapipe as mp
import pyautogui
import math
import collections
import time
import os
import subprocess

# Disable FailSafe
pyautogui.FAILSAFE = False

# --- Configuration ---
```

```

CLICK_THRESHOLD = 0.05 # Distance threshold for pinch
PUSH_THRESHOLD = -0.15 # Z-value threshold for push (negative is closer)
PUSH_HYSTERESIS = 0.05 # Reset buffer for push
SMOOTHING = 0.7 # 0.0 to 1.0

MOUSE_SENSITIVITY = 1.5 # 1.0 = Default. >1.0 = More sensitive.

# --- Terminal Instructions ---
def print_instructions():
    print("="*60)
    print("    WEEK 5: FINAL AI MOUSE CONTROLLER")
    print("="*60)
    print("GESTURE GUIDE:")
    print("1. CURSOR MOVEMENT:")
    print("    - Point with your INDEX FINGER.")
    print("    - The cursor follows your index finger tip.")
    print("")
    print("2. LEFT CLICK:")
    print("    - PINCH your INDEX FINGER and THUMB together.")
    print("    - Short tap for click, hold for drag.")
    print("")
    print("3. RIGHT CLICK:")
    print("    - PINCH your MIDDLE FINGER and THUMB together.")
    print("")
    print("4. FULL SCREENSHOT (Ring+Thumb):")
    print("    - PINCH your RING FINGER and THUMB together.")
    print("    - Saves FULL SCREEN as 'full_screenshot_TIMESTAMP.png' to")
    print("DESKTOP.")
    print("")
    print("5. PUSH CLICK (Experimental):")
    print("    - Move your open hand quickly TOWARDS the camera.")
    print("    - Useful for 'air tapping'.")
    print("="*60)
    print("Press 'q' to quit the application.")
    print("="*60)
    time.sleep(2)

def calculate_distance(p1, p2):
    return math.hypot(p1.x - p2.x, p1.y - p2.y)

def main():
    print_instructions()

    mp_hands = mp.solutions.hands
    mp_drawing = mp.solutions.drawing_utils
    screen_width, screen_height = pyautogui.size()
    cap = cv2.VideoCapture(0)

    # State variables
    left_click_active = False
    right_click_active = False
    push_active = False

    # Smoothing variables
    prev_x, prev_y = 0, 0

    # Depth history for push

```



```

y_mapped = max(0, min(1, y_mapped))

raw_x = int(x_mapped * screen_width)
raw_y = int(y_mapped * screen_height)

cur_x = prev_x + (raw_x - prev_x) * SMOOTHING
cur_y = prev_y + (raw_y - prev_y) * SMOOTHING

try:
    pyautogui.moveTo(cur_x, cur_y)
except pyautogui.FailSafeException:
    pass

prev_x, prev_y = cur_x, cur_y

# --- 2. Left Click ---
dist_left = calculate_distance(index, thumb)
if dist_left < CLICK_THRESHOLD:
    cv2.circle(frame, (int(index.x*W),
int(index.y*H)), 15, (0, 255, 0), cv2.FILLED)
    if not left_click_active:
        pyautogui.click()
        print(">> LEFT CLICK triggered")
        left_click_active = True
else:
    left_click_active = False

# --- 3. Right Click ---
dist_right = calculate_distance(middle, thumb)
if dist_right < CLICK_THRESHOLD:
    cv2.circle(frame, (int(middle.x*W),
int(middle.y*H)), 15, (0, 0, 255), cv2.FILLED)
    if not right_click_active:
        pyautogui.rightClick()
        print(">> RIGHT CLICK triggered")
        right_click_active = True
else:
    right_click_active = False

# --- 4. Screenshot ---
dist_ring = calculate_distance(ring, thumb)
screenshot_state = getattr(main, "screenshot_state",
False)

if dist_ring < CLICK_THRESHOLD:
    cv2.circle(frame, (int(ring.x*W), int(ring.y*H)),
15, (255, 255, 0), cv2.FILLED)
    if not screenshot_state:
        timestamp = int(time.time())
        desktop_path =
os.path.join(os.path.expanduser("~"), "Desktop")
        filename = f"full_screenshot_{timestamp}.png"
        filepath = os.path.join(desktop_path,
filename)

        try:
            subprocess.run(["screencapture", "-x",
filepath], check=True)

```

```

        print(f">> FULL SCREENSHOT SAVED:
{filepath}")
        cv2.putText(frame, "SCREENSHOT SAVED!",
                    (W//2 - 150, H//2),
                    cv2.FONT_HERSHEY_SIMPLEX, 1,
                    (255, 255, 0), 3)
    except Exception as e:
        print(f"Error taking screenshot: {e}")
        screenshot_state = True
        main.screenshot_state = True
    else:
        main.screenshot_state = False

    # --- 5. Push Gesture ---
    current_z = index.z
    z_history.append(current_z)
    avg_z = sum(z_history) / len(z_history)
    bar_h = int(abs(avg_z) * 500)
    cv2.rectangle(frame, (W-30, H-bar_h), (W-10, H),
(255, 0, 0), -1)

    if avg_z < PUSH_THRESHOLD:
        if not push_active:
            if not left_click_active and not
right_click_active:
                pyautogui.click()
                print(f">> PUSH CLICK triggered (Z:
{avg_z:.3f})")
                cv2.putText(frame, "PUSH!", (W//2,
H//2), cv2.FONT_HERSHEY_SIMPLEX, 2, (0,255,255), 3)
                push_active = True
            else:
                if avg_z > (PUSH_THRESHOLD + PUSH_HYSTERESIS):
                    push_active = False

        cv2.imshow('Final AI Mouse', frame)
        if cv2.waitKey(5) & 0xFF == ord('q'):
            break
        cap.release()
        cv2.destroyAllWindows()

if __name__ == "__main__":
    main()

```